DALI ServUI

Dokumentation

Version: 1.0.0

Datum: 2026-05-07

Autor: c42u

Co-Autor: ClaudeCode

Lizenz: GPLv3

Inhaltsverzeichnis

1. Glossar
2. Disclaimer
3. Ausgangslage und Projektidee
4. Verwendete Software
5. Architektur
6. Unterstützte Hardware
7. Installation
8. Konfiguration
9. Bedienung
10. REST-API
11. Server-Sent Events (SSE)
12. Multi-Dashboard
13. Commissioning
14. Bus-Protokoll und Sniffer
15. Sicherheit
16. Backup und Wiederherstellung
17. Fehlermanagement
18. Was ist neu in v1.0.0
19. Anhang A – Checkliste

1. Glossar

Begriff	Beschreibung
DALI	Digital Addressable Lighting Interface – IEC 62386, Standard für digitale Lichtsteuerung
DALI-Bus	Zweidraht-Kommunikationsleitung (DALI+ / DALI-), verpolungssicher, max. 64 Geräte
Forward Frame	16-Bit-Befehl vom Master an die Geräte (1 Startbit + 8 Bit Adresse + 8 Bit Befehl)
Backward Frame	8-Bit-Antwort vom Gerät an den Master
Kurzadresse	Eindeutige Geräteadresse 0–63 auf dem DALI-Bus

Begriff	Beschreibung
Commissioning	Prozess der automatischen Adresszuweisung an DALI-Geräte
DAPC	Direct Arc Power Control – Helligkeitssteuerung (0–254)
Manchester-Encoding	Leitungscode bei dem jedes Bit durch einen Pegelwechsel dargestellt wird
HID	Human Interface Device – USB-Geräteklasse für Eingabegeräte
FTDI	Future Technology Devices International – Hersteller von USB-zu-Seriell-Wandlern
mikroBUS	Standardisierter Steckverbinder von MikroElektronika für Click-Boards
Dryrun	Testmodus ohne angeschlossene Hardware

2. Disclaimer

Sicherheitshinweis und Haftungsausschluss

Die Nutzung von DALI ServUI erfolgt **auf eigene Gefahr**. Der Autor übernimmt keine Haftung für Personen-, Sach- oder Vermögensschäden, die durch den Einsatz dieser Software entstehen. Diese Software wurde mit Unterstützung von KI-Technologie (Claude, Anthropic) entwickelt.

Installation und Inbetriebnahme

DALI ServUI steuert elektrische Beleuchtungsanlagen. Die Installation, Verkabelung und Inbetriebnahme des DALI-Bus-Systems – insbesondere alle Arbeiten an der Netzspannungsseite der Vorschaltgeräte – sind ausschließlich durch **fachkundiges, ausgebildetes Personal** (Elektrofachkraft) durchzuführen. Die jeweils geltenden nationalen Vorschriften (z. B. DIN VDE 0100, ÖVE/ÖNORM E 8001, NIN) sind zu beachten. Unsachgemäße Konfiguration oder fehlerhafte Verkabelung kann zu Fehlfunktionen der Beleuchtung, zu Geräteschäden oder im schlimmsten Fall zu Personenschäden führen.

Lizenz

GNU General Public License v3.0 (GPLv3). Die vollständige Lizenz ist unter https://www.gnu.org/licenses/gpl-3.0.html einsehbar.

3. Ausgangslage und Projektidee

Ausgangslage

Im bestehenden Projekt **DALIPi** wurde eine CLI-basierte DALI-Lichtsteuerung über den Hasseb USB DALI Master realisiert. Die Steuerung erfolgte per Kommandozeile und über eine TCP-Bridge mit zeilenbasiertem ASCII-Protokoll. Parallel existiert das Open-Source-Projekt **daliserver** (github.com/onitake/daliserver), ein C-basierter TCP-Daemon für den Tridonic DALI USB-Adapter.

Beide Ansätze hatten Einschränkungen: kein Web-Interface, keine Containerisierung, eingeschränkte Hardware-Unterstützung.

Projektidee

DALI ServUI kombiniert die Stärken beider Projekte zu einer modernen, containerisierten Lösung mit Web-Oberfläche:

- **Multi-Hardware-Support:** Hasseb USB, MikroE DALI Click (GPIO + USB), Testmodus
- **Web-UI:** Responsive Oberfläche mit Dashboard, Geräte-, Gruppen- und Discovery-Ansichten
- **REST-API:** Vollständige API für Integration mit anderen Systemen
- **Docker + nativ:** Betrieb als Container oder direkte Linux-Installation
- **Zweisprachig:** Deutsch und Englisch

Die Architektur orientiert sich am daliserver-Ansatz (Queue-basiertes Command-Multiplexing) und wurde in Python mit Flask + Jinja2 umgesetzt.

4. Verwendete Software

Software	Version	Quelle	Zweck
Python	3.12	python.org	Programmiersprache
Flask	3.x	pypi.org/project/Flask	Web-Framework
Gunicorn	22.x	pypi.org/project/gunicorn	WSGI-Server
hidapi	0.14.x	pypi.org/project/hidapi	USB-HID-Zugriff (Hasseb)

Software	Version	Quelle	Zweck
gpiod	2.x	pypi.org/project/gpiod	GPIO-Zugriff (MikroE/RPi)
pyftdi	0.55.x	pypi.org/project/pyftdi	FTDI-USB-Zugriff (MikroE)
Docker	27.x	docker.com	Containerisierung
Jinja2	3.x	(in Flask enthalten)	Template-Engine

5. Architektur

Schichtenmodell

```
┌──────────────────────────────────────────────────┐
│  Web-Browser (Dashboard, Steuerung, Hilfe)       │
└──────────────────────────────────────────────────┘
                │
                │ HTTP / REST-API
┌───────────────▼──────────────────────────────────┐
│  Flask + Jinja2 (main.py)                        │
│  - Web-Routes + Templates                        │
│  - REST-API v1                                   │
│  - Translations (DE/EN)                          │
└──────────────────────────────────────────────────┘
                │
                │
┌───────────────▼──────────────────────────────────┐
│  DALI Service (dali_service.py)                  │
│  - Queue-basiertes Command-Multiplexing          │
│  - Worker-Thread (sequentielle Abarbeitung)      │
│  - High-Level DALI-Befehle                       │
│  - Bus-Scan + Commissioning                      │
└──────────────────────────────────────────────────┘
                │
                │ Plugin-Interface
┌───────────────▼──────────────────────────────────┐
│  Treiber-Registry (drivers/)                     │
│  ├── hasseb.py      (USB-HID)                    │
│  ├── mikroe_gpio.py (Raspberry Pi GPIO)          │
│  ├── mikroe_ftdi.py (FTDI FT2232H USB)           │
│  └── dryrun.py      (Testmodus)                  │
└──────────────────────────────────────────────────┘
                │
                │
┌───────────────▼──────────────────────────────────┐
│  DALI-Hardware (Bus, Leuchten, Vorschaltgeräte)  │
└──────────────────────────────────────────────────┘
```

Plugin-Treiber-Architektur

Alle Treiber erben von der abstrakten Basisklasse `DaliDriver` und implementieren:

- `open()` – Hardware-Verbindung öffnen
- `close()` – Hardware-Verbindung schließen
- `send_frame()` – DALI Forward Frame senden, optional auf Antwort warten
- `get_info()` – Treiber-Informationen für die WebUI

Die Registry (`drivers/registry.py`) registriert Treiber per Lazy Import – fehlende Dependencies (z.B. `gpiod` auf einem PC ohne GPIO) werden toleriert. Der Treiber wird dann als „nicht verfügbar" markiert.

Datei-Übersicht

Datei	Funktion
`Code/app/main.py`	Flask-Hauptanwendung, Routes, API
`Code/app/dali_service.py`	DALI-Service mit Queue und High-Level-Befehlen
`Code/app/config.py`	Konfiguration (Umgebungsvariablen)
`Code/app/translations.py`	Übersetzungen DE/EN
`Code/app/drivers/base.py`	Abstrakte Treiber-Basisklasse
`Code/app/drivers/hasseb.py`	Hasseb USB-HID-Treiber
`Code/app/drivers/mikroe_gpio.py`	MikroE GPIO-Treiber (RPi)
`Code/app/drivers/mikroe_ftdi.py`	MikroE FTDI-Treiber (USB)
`Code/app/drivers/dryrun.py`	Simulierter Testmodus
`Code/app/drivers/registry.py`	Treiber-Registry
`Code/app/templates/`	Jinja2-Templates (6 Seiten)
`Code/app/static/`	CSS + JavaScript

6. Unterstützte Hardware

Hasseb USB DALI Master

- **Hersteller:** Hasseb (hasseb.fi)
- **Schnittstelle:** USB-HID (Plug & Play, keine Treiber nötig)

- **USB:** Vendor 0x04CC, Product 0x0802
- **Bus-Stromversorgung:** Integriert, 250mA max.
- **DALI-Version:** DALI 2 kompatibel
- **Protokoll:** 10-Byte HID-Pakete mit Sequenznummern

MikroElektronika DALI Click *(in Vorbereitung – Coming Soon)*

> ***Hinweis:*** *Die Anbindung der MikroE-Boards (sowohl GPIO als auch FTDI/USB) ist in v0.8.0 als Coming Soon gekennzeichnet. Die Treiber-Stubs sind im Code enthalten, der Hardware-Test mit echten EVGs läuft jedoch noch (Optokoppler-Pegel bei 3.3V grenzwertig). Im Settings-Dialog werden diese Treiber daher mit einem „Coming Soon"-Stempel markiert und können über die UI nicht aktiviert werden.*

- **Hersteller:** MikroElektronika (mikroe.com)
- **Schnittstelle:** GPIO (TX=RST, RX=INT auf mikroBUS)
- **Bus-Stromversorgung:** Extern erforderlich
- **TX-Logik:** Nicht invertiert (TX HIGH = Bus inaktiv)
- **Anschluss:** Pi Click Shield oder Click USB Adapter

MikroElektronika DALI 2 Click *(in Vorbereitung – Coming Soon)*

- **Hersteller:** MikroElektronika (mikroe.com)
- **Schnittstelle:** GPIO (TX=RST, RX=INT auf mikroBUS)
- **Bus-Stromversorgung:** Extern erforderlich
- **TX-Logik:** Invertiert (TX HIGH = Bus aktiv) → Option „TX invertiert" aktivieren!
- **DALI-Version:** DALI 2 kompatibel
- **Anschluss:** Pi Click Shield oder Click USB Adapter

Anschlussoptionen für MikroE-Boards

Methode	Hardware	Python-Library
Raspberry Pi direkt	Pi Click Shield (connectors soldered)	gpiod
USB an jedem PC	Click USB Adapter (FT2232H)	pyftdi

7. Installation

Option A: Docker

```
# Repository klonen
git clone https://git.zucker.network/zucker/public-apps/dali-servui.git
cd dali-servui

# Container starten
cd Docker/
docker compose up -d
```

Der Container ist unter `http://<IP>:5000` erreichbar.

Für USB-Hardware muss der Container Zugriff auf das USB-Gerät haben. In der `docker-compose.yml` ist `privileged: true` voreingestellt. Alternativ kann ein spezifisches Device gemappt werden:

```
devices:
  - /dev/hidraw0:/dev/hidraw0
```

Option B: Direkte Linux-Installation

```
# Installationsskript ausführen
sudo ./Code/install.sh install
```

Das Skript führt folgende Schritte aus:

1. Systemabhängigkeiten installieren (python3, libhidapi, libusb)
2. Benutzer `dali` und Gruppe `dali` anlegen
3. udev-Regel für USB-Zugriff einrichten
4. Python venv erstellen und Dependencies installieren
5. systemd-Service `dali-servui.service` einrichten

```
# Service starten
sudo systemctl start dali-servui

# Status prüfen
sudo systemctl status dali-servui

# Logs anzeigen
sudo journalctl -u dali-servui -f

# Service deaktivieren
sudo systemctl stop dali-servui
```

Deinstallation

```
sudo ./Code/install.sh uninstall
```

Daten im Verzeichnis `/var/lib/dali-servui` und der Benutzer `dali` bleiben erhalten.

8. Konfiguration

Umgebungsvariablen

Variable	Beschreibung	Standard
`DALI_DRIVER`	Treiber-ID: hasseb, mikroe_gpio, mikroe_ftdi, dryrun	(gespeicherte Konfig)
`DALI_HOST`	Bind-Adresse des Webservers	0.0.0.0
`DALI_PORT`	Port des Webservers	5000
`DALI_LANG`	Standard-Sprache (de/en)	de
`DALI_API_TOKEN`	API-Authentifizierungstoken (leer = deaktiviert)	
`DALI_LOG_LEVEL`	Log-Level (DEBUG, INFO, WARNING, ERROR)	INFO
`DALI_LOG_FILE`	Pfad zur Log-Datei (leer = nur stdout)	
`DALI_DATA_DIR`	Datenverzeichnis für Konfiguration und Backup	Code/data
`DALI_SECRET_KEY`	Flask-Session-Schlüssel	leer → Auto-Generierung in `data/secret_key` (0600)

Variable	Beschreibung	Standard
`DALI_CORS_ORIGINS`	Komma-getrennte CORS-Origins; `*` für alle	leer (kein CORS-Header)

Treiber-Konfiguration über WebUI

Die Treiber-Konfiguration kann direkt in den Einstellungen der Web-Oberfläche vorgenommen werden. Die Konfiguration wird als `driver_config.json` im Datenverzeichnis gespeichert und beim nächsten Start automatisch geladen.

udev-Regel (Hasseb)

Die Installationsroutine richtet automatisch eine udev-Regel ein:

```
KERNEL=="hidraw*", ATTRS{idVendor}=="04cc", ATTRS{idProduct}=="0802", MODE="0660",
GROUP="dali"
```

9. Bedienung

Dashboard

Das Dashboard zeigt Verbindungsstatus, Firmware-Version, Geräteanzahl und Warteschlangengröße. Im unteren Bereich steuern Sie per Broadcast alle Leuchten gleichzeitig. Die Geräte-Karten zeigen den aktuellen Status jedes einzelnen Geräts.

Geräte-Seite

Tabellarische Übersicht aller erkannten DALI-Geräte mit Adresse, Helligkeit (Slider), Gerätetyp, Gruppenzugehörigkeit und Erreichbarkeitsstatus.

Gruppen-Seite

Zeigt alle DALI-Gruppen (0–15) mit ihren Mitgliedern. Jede Gruppe kann gemeinsam ein-/ausgeschaltet oder in der Helligkeit geregelt werden.

Geräte-Erkennung

- **Bus-Scan:** Prüft Adressen 0–63 (~10–20 Sekunden)
- **Commissioning (alle):** Setzt alle Adressen zurück und vergibt neue
- **Commissioning (nur neue):** Adressiert nur Geräte ohne Kurzadresse

Einstellungen

Treiber-Auswahl mit Konfigurationsfeldern. Der Wechsel erfolgt sofort und wird dauerhaft gespeichert. Sprachwechsel zwischen Deutsch und Englisch.

Hilfe

Integrierte Hilfe-Seite mit vollständiger Anleitung zu allen Features, Hardware-Konfiguration, API-Dokumentation und Fehlerbehebung.

Buy me a coffee

Direkt rechts neben „Hilfe" in der Navbar führt der Link zu buymeacoffee.com/c42u – freiwilliger Beitrag zur Weiterentwicklung. Öffnet sich in einem neuen Tab und hat keine Auswirkung auf die Funktionalität.

10. REST-API

Basis-URL: `http://<host>:5000/api/v1/`

Bei konfiguriertem API-Token: Header `X-API-Token: <token>` mitsenden.

Status

Methode	Pfad	Beschreibung
GET	`/status`	Service-Status (running, connected, driver, firmware, devices)
GET	`/devices`	Alle erkannten Geräte
GET	`/drivers`	Verfügbare Treiber
GET	`/drivers/config`	Aktuelle Treiber-Konfiguration

Geräte-Steuerung

Methode	Pfad	Body	Beschreibung
POST	`/devices/{addr}/on`	–	Gerät einschalten
POST	`/devices/{addr}/off`	–	Gerät ausschalten

Methode	Pfad	Body	Beschreibung
POST	`/devices/{addr}/level`	`{"level": 0..254}`	Helligkeit setzen

Broadcast

Methode	Pfad	Body	Beschreibung
POST	`/broadcast/on`	–	Alle einschalten
POST	`/broadcast/off`	–	Alle ausschalten
POST	`/broadcast/level`	`{"level": 0..254}`	Alle Helligkeit

Gruppen

Methode	Pfad	Body	Beschreibung
POST	`/groups/{g}/on`	–	Gruppe einschalten
POST	`/groups/{g}/off`	–	Gruppe ausschalten
POST	`/groups/{g}/level`	`{"level": 0..254}`	Gruppen-Helligkeit

Discovery & Treiber

Methode	Pfad	Body	Beschreibung
POST	`/scan`	–	Bus-Scan starten
POST	`/commission`	`{"broadcast": true}`	Commissioning
POST	`/raw`	`{"address":0,"command":0,"expect_reply":false}`	Roher DALI-Befehl
POST	`/drivers/switch`	`{"driver_id":"hasseb"}`	Treiber wechseln

Methode	Pfad	Body	Beschreibung
GET	`/drivers/commission-status`	–	Commissioning-Status + Log-Snapshot
GET	`/drivers/commission-stream`	–	Live-SSE-Stream beim Commissioning
POST	`/drivers/stop` / `/start`	–	Treiber stoppen / starten (USB freigeben)
POST	`/sniff`	`{"duration": 10}`	Bus-Sniffer (passiv lauschen, max. 60s)
POST	`/reset-addresses`	–	Alle Kurzadressen löschen
POST	`/factory-reset`	–	Vollständiger DALI Factory Reset

Labels und Gruppen-Verwaltung

Methode	Pfad	Body	Beschreibung
GET	`/labels`	–	Alle Geräte- und Gruppennamen
POST	`/labels/device/{addr}`	`{"name": "..."}`	Gerätename setzen (max. 100 Zeichen)
POST	`/labels/group/{g}`	`{"name": "..."}`	Gruppenname setzen (max. 100 Zeichen)
GET	`/groups`	–	Alle Gruppen mit Namen, Mitgliedern, Level

Methode	Pfad	Body	Beschreibung
POST	`/devices/{addr}/groups`	`{"group":0..15}`	Zu Gruppe hinzufügen
DELETE	`/devices/{addr}/groups`	`{"group":0..15}`	Aus Gruppe entfernen

DT8 Tunable White & RGB

Methode	Pfad	Body	Beschreibung
POST	`/devices/{addr}/colour-temp`	`{"kelvin":3000}` oder `{"mirek":333}`	Farbtemperatur
POST	`/devices/{addr}/rgb`	`{"r":0,"g":0,"b":0}`	RGB-Farbe (DT8)
POST	`/broadcast/colour-temp`	wie oben	Broadcast Tunable White
POST	`/broadcast/rgb`	wie oben	Broadcast RGB
GET	`/features`	–	Aktuelle Feature-Flags
POST	`/features`	`{"dt6":true,"dt8_tc":true,"dt8_rgb":false}`	Features umschalten

Multi-Dashboard

Methode	Pfad	Body	Beschreibung
GET	`/dashboards`	–	Alle Dashboards
GET	`/dashboards/{id}`	–	Einzelnes Dashboard

Methode	Pfad	Body	Beschreibung
POST	`/dashboards`	`{"name":"...","items":[...]}`	Neues Dashboard erstellen
PUT	`/dashboards/{id}`	–	Dashboard aktualisieren
DELETE	`/dashboards/{id}`	–	Dashboard löschen (default geschützt)

Bus-Protokoll und Echtzeit

Methode	Pfad	Body	Beschreibung
GET	`/buslog`	`?limit=100&since=...`	Bus-Frames abrufen
DELETE	`/buslog`	–	Bus-Protokoll löschen
POST	`/buslog/toggle`	`{"enabled": true}`	Bus-Logging ein/aus
GET	`/events`	–	SSE-Stream aller Zustandsänderungen

11. Server-Sent Events (SSE)

Für Echtzeit-Updates ohne Polling steht der SSE-Endpunkt `GET /api/v1/events` zur Verfügung. Der Stream liefert JSON-Events bei jeder Zustandsänderung:

Event-Typ	Felder	Beschreibung
`level`	`address`, `level`	Helligkeit eines Geräts geändert
`on` / `off`	`address`	Gerät ein-/ausgeschaltet
`group_level`	`group`, `level`	Gruppen-Helligkeit
`group_on` / `group_off`	`group`	Gruppe ein-/ausgeschaltet
`scan_complete`	`count`, `devices`	Bus-Scan abgeschlossen
`colour_temp`	`address`, `mirek`	DT8 Tunable White
`rgb`	`address`, `r`, `g`, `b`	DT8 RGB

Keepalive-Kommentare alle 30 Sekunden verhindern Proxy-Timeouts. Mehrere parallele Clients sind erlaubt (Thread-sicherer Event-Bus).

Beispiel (Browser):

```
const es = new EventSource('/api/v1/events');
es.onmessage = (e) => console.log(JSON.parse(e.data));
```

12. Multi-Dashboard

Mehrere Dashboards können pro Raum oder Anwendungsfall konfiguriert werden. Jedes Dashboard ist eine kuratierte Auswahl aus Geräten und Gruppen mit optionalen Statuskarten und Broadcast-Steuerung.

Item-Typen

- `all_devices` – alle erkannten Geräte
- `all_groups` – alle Gruppen
- `device` mit `address` (0..63) – einzelnes Gerät
- `group` mit `id` (0..15) – einzelne Gruppe

Items werden serverseitig validiert; ungültige Adressen oder Gruppen-IDs werden verworfen.

Persistenz

Die Konfiguration liegt in `dashboards.json` im Daten-Verzeichnis. Das `default`-Dashboard ist geschützt und kann weder umbenannt noch gelöscht werden.

13. Commissioning

Das Commissioning vergibt eindeutige Kurzadressen an alle DALI-Geräte am Bus. DALI ServUI nutzt für Hasseb das python-dali Subprocess-Verfahren, das mit über 20 EVGs stabil getestet wurde.

Ablauf

1. Treiber wird automatisch gestoppt (USB freigegeben)
2. Alle Kurzadressen werden gelöscht (`Initialise` + `DTR0(0xFF)` + `SetShortAddress(Broadcast)`)
3. Commissioning-Sequenz vergibt neue Adressen
4. Bus-Scan validiert das Ergebnis
5. Treiber wird neu gestartet, UI zeigt Geräte sofort an

Schutz vor Doppel-Ausführung

- Nur ein Commissioning gleichzeitig (`threading.Lock`)
- HTTP 409 wenn ein zweiter Start versucht wird
- Live-Output via SSE (`/drivers/commission-stream`)
- Output bleibt bei Seitenwechsel im Backend gepuffert (`/drivers/commission-status`) – Reload zeigt vorherigen Verlauf

CLI

`Code/dali_commission.py` führt das gleiche Verfahren standalone aus (`--reset-only` , `--scan-only` , `--no-reset` als Optionen).

14. Bus-Protokoll und Sniffer

Das Bus-Protokoll zeichnet alle TX/RX-Frames mit Timestamp und lesbarer Beschreibung in einem Ringbuffer (500 Einträge) auf. Für reines Mitlesen ohne eigenen Master-Verkehr steht der Hasseb-Sniffer-Modus bereit.

Aspekt	Details
Frame-Beschreibungen	DAPC, DTR0/DTR1, Group-Befehle, Commissioning-Frames, DT6/DT8
Aktivierung	Standard ein, abschaltbar via `POST /buslog/toggle`
Sniffer-Dauer	0,5 s bis max. 60 s, von API geclamped
Anzeige	`/buslog` -Seite mit Auto-Refresh, Pause, Clear

Eine direkte Frame-Eingabe ist über `POST /api/v1/raw` möglich (`address` und `command` jeweils 0..255).

15. Sicherheit

Authentifizierung und CORS

Aspekt	Default	Empfehlung
API-Token	leer (Auth aus)	für LAN-Betrieb gesetzt; alle Clients senden `X-API-Token`

Aspekt	Default	Empfehlung
CORS-Origins	leer (kein Header)	präzise Origins (z. B. Home-Assistant-URL) statt `*`
Bind-Adresse	`0.0.0.0`	bei Reverse-Proxy auf `127.0.0.1` setzen, sonst Firewall vor Port 5000
TLS	nicht eingebaut	Reverse-Proxy (Caddy, Traefik, nginx) davor

Wenn weder API-Token gesetzt noch der Port abgeschottet ist, kann jede Webseite, die ein Anwender im selben Browser öffnet, per CORS oder schlicht über CSRF-fähige `POST`-Requests an die DALI-Lampen schicken. Mindestens eines der beiden Schutzmittel sollte aktiv sein.

Frontend-Authentifizierung (seit v0.9.0)

Bei gesetztem `DALI_API_TOKEN` schreibt das Backend den Token in ein verstecktes `<meta name="x-api-token">` der gerenderten Seite. Das mitgelieferte JavaScript liest ihn beim Laden und sendet ihn als `X-API-Token`-Header an jeden API-Request. SSE-Verbindungen (`/api/v1/events`, `/api/v1/drivers/commission-stream`) authentifizieren über `?token=...`-Query-Parameter, weil `EventSource` keine Custom-Header senden kann.

> ***Bedrohungsmodell-Hinweis:*** *Der Token landet im DOM und ist damit für jeden Browser-User sichtbar, der die UI aufrufen kann. Das ist bewusst so. API-Token = „Maschinen- oder UI-Authentifizierung". Wer die UI selbst vor Browser-Zugriff schützen will, schaltet einen Reverse-Proxy mit Basic-Auth oder OIDC davor – der Token bleibt trotzdem für externe API-Clients (Home Assistant) wirksam.*

HTTP-Sicherheitsheader (seit v0.9.0)

Alle HTTP-Antworten enthalten:

Header	Wert
`X-Frame-Options`	`DENY`
`X-Content-Type-Options`	`nosniff`
`Referrer-Policy`	`same-origin`

Header	Wert
`Content-Security-Policy`	`default-src 'self'; img-src 'self' data:; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; connect-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-action 'self'`

`'unsafe-inline'` ist nötig, solange Templates Inline- `onclick`-Handler und `<script>`-Blöcke verwenden. Eine spätere Migration auf `addEventListener` würde das ablösen.

Ressourcen-Caps

Aspekt	Cap	Wirkung
Bus-Log	`MAX_BUSLOG_SIZE = 500` Einträge	Ringbuffer, älteste Einträge fallen raus
SSE-Queue je Client	`maxsize = 50` Events	Langsame Clients fliegen aus dem Listener-Pool
SSE-Listener gesamt	`MAX_SSE_LISTENERS = 50` (seit v0.9.0)	Anti-DoS: 51. `/api/v1/events`-Aufruf erhält `503`
JSON-Loader	`MAX_JSON_BYTES = 5 MB` (seit v0.9.0)	Loader liefert Default zurück, wenn `data/*.json` aufgebläht wurde

Secrets

- `DALI_SECRET_KEY` wird automatisch generiert und in `data/secret_key` (Mode 0600) abgelegt, wenn die ENV nicht gesetzt ist. Damit bleiben Flask-Sessions auch nach Container-Neustart gültig.
- Die `udev`-Regel beschränkt Zugriff auf `/dev/hidraw*` auf die Gruppe `dali` (Mode 0660).

Health-Endpoint

`GET /healthz` (auth-frei) ist der dedizierte Endpunkt für Container- Healthchecks und Reverse-Proxy-Probes. Er enthält keine Geräteinformationen, nur `{"status":"ok","version":"..."}`.

16. Backup und Wiederherstellung

Backup erstellen

```
# Linux-Installation
./Code/backup.sh /pfad/zum/backup

# Docker
./Code/backup.sh /pfad/zum/backup
```

Das Skript sichert: - Treiber-Konfiguration (`driver_config.json`) - Alle Daten im Datenverzeichnis - Bei Docker: Das Docker-Volume `dali-servui_dali-data`

Backup wiederherstellen

```
# Linux-Installation
sudo ./Code/restore.sh /pfad/zum/backup/dali-servui_backup_YYYYMMDD_HHMMSS.tar.gz

# Docker (Zielverzeichnis angeben)
./Code/restore.sh backup.tar.gz /pfad/zum/docker-volume
```

Die Wiederherstellung stoppt den Service, kopiert die Daten und startet den Service automatisch wieder.

17. Fehlermanagement

Häufige Probleme

Problem	Ursache	Lösung
„Nicht verbunden"	Kein USB-Adapter erkannt	`lsusb` prüfen, udev-Regel prüfen
Bus-Scan findet nichts	Keine Bus-Spannung / keine Adressen	DALI-Bus prüfen, Commissioning starten
Timeout bei Befehlen	Hardware-Kommunikation gestört	USB-Kabel prüfen, Service neu starten
MikroE: keine Reaktion	TX-Invertierung falsch	DALI 2 Click: „TX invertiert" aktivieren
FTDI: Verbindung fehlgeschlagen	Kernel-Treiber blockiert	`sudo rmmod ftdi_sio usbserial`

Problem	Ursache	Lösung
GPIO: Permission denied	Fehlende Berechtigung	Benutzer zur Gruppe `gpio` hinzufügen

Log-Analyse

```
# Live-Logs anzeigen
sudo journalctl -u dali-servui -f

# Nur Fehler anzeigen
sudo journalctl -u dali-servui -p err

# Debug-Modus aktivieren
DALI_LOG_LEVEL=DEBUG sudo systemctl restart dali-servui
```

Dryrun-Modus

Der Testmodus (`DALI_DRIVER=dryrun`) simuliert alle Hardware-Operationen. Queries werden mit `0xFF` beantwortet. Dieser Modus ist nützlich für:

- Test der Web-Oberfläche ohne Hardware
- Entwicklung und Debugging
- Demonstrations- und Schulungszwecke

18. Was ist neu in v1.0.0

v1.0.0 – 2026-05-07 (aktuelle Version)

Erstes öffentliches Release. Funktional auf Stand von 0.9.0; Fokus dieser Version: UI-Bugfixes, Aufräumen der Dokumentation und Versions-Konsistenz über alle Quelldateien hinweg.

- **Bug behoben – Geräte-Status zeigte immer „grün"**: Die Status-Spalte in `/devices` interpretierte das interne `present`-Flag (Bus-Erreichbarkeit beim letzten Scan) als „eingeschaltet". Damit erschien jedes Gerät als „aktiv", auch wenn es ausgeschaltet war. Die Spalte zeigt jetzt drei Zustände als Pill-Badge analog zu den Gruppen-Mitgliedern:
  - **Ein** (grün) – `level > 0`
  - **Aus** (grau) – `level == 0` und am Bus erreichbar
  - **Nicht erreichbar** (orange) – `present == False`
- **Verbesserte Sichtbarkeit des Schaltzustands**: Die Tabellenzeile bekommt zusätzlich einen dezenten Indikator (linker Farbbalken plus leichter Hintergrund), gleiche visuelle Sprache wie bei den Gruppen-Karten (`group-on/off/warning`).

- **Live-Update ohne Reload**: Die JS-Funktionen `deviceOn`, `deviceOff` und `deviceLevel` synchronisieren Badge, Zeilenklasse und Slider-Wert-Anzeige direkt nach dem API-Aufruf (neue Hilfsfunktion `updateDeviceRow`).
- **Navbar „Kaffee?" → „Kaffee"** (DE), „Coffee?" → „Coffee" (EN).
- **Versions-Konsistenz**: Alle 21 Header in Python-, HTML-, CSS-, JS-, Bash- und Docker-Dateien wurden auf Version 1.0.0 (Deploy 2026-05-07) vereinheitlicht. Vorher trugen die Dateien Stände zwischen 0.1.0 und 0.6.0, abhängig vom Zeitpunkt der letzten Änderung.
- **Dokumentation aufgeräumt**: Alte Doku- und Reviewbericht-Stände (v0.2.0, v0.8.0, v0.9.0) wurden aus `Dokumentation/` und `Reviewbericht/` nach `Versionssicherung/v<X.Y.Z>/` verschoben. In den Hauptordnern liegt nur noch der aktuelle Stand.
- **README-Überarbeitung**: Feature-Liste auf Stand 1.0.0, Hardware- Tabelle mit Status-Spalte (produktiv vs. in Vorbereitung), vollständige Tabelle der ENV-Variablen, aktualisierte API-Übersicht.

v0.9.0 – 2026-05-05

- **HTTP-Sicherheitsheader**: `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Content-Security-Policy`
- **SSE-Listener-Cap** `MAX_SSE_LISTENERS = 50` mit `503`-Antwort bei Erreichen
- **JSON-Loader-Härtung**: 5 MB Größencap, typisierte Whitelist für `load_driver_config`
- **Frontend-Auth**: Token via `<meta name="x-api-token">`; JS sendet `X-API-Token`-Header; `EventSource` nutzt `?token=...`-Query
- **Buy-me-a-coffee-Link** in der Navbar neben „Hilfe"

v0.8.0 – 2026-05-05

- **MikroE Coming-Soon-Stempel** im Settings-Dialog auf den Treiber-Cards
- **Healthz-Endpoint** `GET /healthz` (auth-frei) für Container-Healthchecks
- **SECRET_KEY-Auto-Generierung** persistent in `data/secret_key` (0600)
- **CORS-Origins via ENV** `DALI_CORS_ORIGINS` (Komma-Liste, Default: leer)
- **Compose-Bug behoben**: doppelter `volumes:`-Block, USB-Mapping als Default
- **Härtung**: Label-Längen-Cap, Dashboard-Item-Schema-Validierung, Sniff-Duration-Floor
- **Thread-sichere Commissioning-Log-Operationen**, Signal-Handler in App-Factory
- LICENSE-Datei (GPLv3-Volltext) ergänzt

v0.7.0 – 2026-04-04

Multi-Dashboard pro Raum, REST-API für Dashboards, wiederverwendbare Group-Card-Partials, Default-Dashboard geschützt.

v0.6.0 – 2026-04-04

Gruppen-API mit Mitgliedern und Level, CORS-Header für API, SSE-Stream `/api/v1/events` mit Keepalive.

v0.5.x – 2026-04-04

Commissioning mit Live-Stream (SSE), python-dali Subprocess für Hasseb, CLI-Skript `dali_commission.py`, Bus-Sniffer, Treiber-Stop/Start-API, Adress-Reset und Factory-Reset.

v0.4.x – 2026-04-03

Bus-Protokoll mit Ringbuffer und lesbaren Frame-Beschreibungen, Gruppenzuweisung per UI (AddToGroup/RemoveFromGroup), persistente Labels für Geräte und Gruppen, Lucide-Icons in der Navigation.

v0.3.x – 2026-03-30

DT6/DT8 (Tunable White, RGB), Feature-Toggles, erweiterte API, zweisprachige Hilfeseite mit TOC, integrierter Reverse-Proxy-Hinweis.

19. Anhang A – Checkliste

Installations-Checkliste

- ☐ Hardware angeschlossen und erkannt (`lsusb` oder `gpioinfo`)
- ☐ Docker oder Linux-Installation abgeschlossen
- ☐ Web-UI erreichbar unter `http://<IP>:5000`
- ☐ Treiber in Einstellungen ausgewählt und aktiviert
- ☐ Bus-Scan durchgeführt, Geräte erkannt
- ☐ Einzelsteuerung (Ein/Aus/Level) funktioniert
- ☐ Gruppensteuerung funktioniert
- ☐ Broadcast-Steuerung funktioniert

Sicherheits-Checkliste

- ☐

`DALI_SECRET_KEY` gesetzt **oder** persistierter `data/secret_key` ist vorhanden und 0600 gesichert
- ☐ `DALI_API_TOKEN` gesetzt (falls API über das LAN erreichbar)
- ☐ `DALI_CORS_ORIGINS` auf bekannte Frontend-Hosts beschränkt (kein `*`)
- ☐ Zugriff auf Port 5000 per Firewall eingeschränkt
- ☐ HTTPS via Reverse-Proxy (z. B. Caddy, Traefik) konfiguriert
- ☐ Healthcheck nutzt `/healthz` (auth-frei) – Container bleibt healthy

Hardware-Checkliste

- ☐ DALI-Bus-Spannung gemessen (typ. 16V, min. 11.5V, max. 22.5V)
- ☐ Maximale Bus-Last beachtet (250mA bei Hasseb, extern bei MikroE)
- ☐ Maximale Leitungslänge beachtet (300m bei 1.5mm², 100m bei 0.5mm²)
- ☐ Alle Geräte haben Kurzadressen (Commissioning durchgeführt)